

SD-01 · L0 Self-Serve KB Chatbot — AI Notes

RAG-powered support chatbot with citations and confidence-based escalation

1. What It Does

An AI chatbot that answers support queries by searching 4 Markdown KB articles (16+ sections). It scores its own confidence, shows citations, and auto-escalates low-confidence answers as tickets.

2. Architecture Flow

Step	Component	Action
1	Frontend (index.html)	User types a support question
2	RAG Layer	All 4 KB articles injected into Claude prompt
3	Claude API	Searches KB, generates answer + confidence score
4	Response Engine	Routes by confidence: answer / warn / escalate
5	Ticket System	Auto-creates ticket with context if confidence < 0.55

3. Confidence Scoring

Level	Score	Action	UI
High	≥ 0.75	Answer + Citations	Green bar
Medium	0.55–0.74	Answer + Caution warning	Yellow bar
Low	< 0.55	Auto-escalate → Ticket	Red bar, ticket form opens

4. Knowledge Base Articles

File	Topics Covered
account_management.md	Password reset, profile update, 2FA, account deletion
billing_payments.md	Invoices, payment methods, refunds, plan upgrades
technical_support.md	Login errors, crashes, connectivity, browser issues
getting_started.md	Onboarding, first-time setup, feature tour, FAQs

5. Tech Stack

Layer	Technology	Purpose
Frontend	Vanilla HTML/CSS/JS	Zero dependencies, single file, runs instantly
AI Model	Claude Sonnet 4.6	RAG accuracy, Anthropic API
KB Format	Markdown files	Easy to edit, human-readable, Git-friendly
Backend	FastAPI (optional)	Lightweight Python REST API
Testing	Pytest	Happy-path unit tests

SD-01 · AI Notes (continued)

6. Sample API Response (JSON)

```
{ "answer": "To reset your password, click Forgot Password on the login page...",
  "confidence": 0.92,
  "citations": [{ "article": "Account Management", "section": "How to Reset Your Password" }],
  "topic": "account", "should_escalate": false }
```

7. File Structure

```
kb-chatbot/
■■■ index.html ← Full frontend + RAG + dashboard (open directly)
■■■ src/app.py ← FastAPI backend (optional)
■■■ src/rag_engine.py ← RAG search & confidence logic
■■■ src/ticket_service.py ← Ticket management
■■■ data/kb_articles/ ← 4 Markdown KB files
■■■ outputs/sample_output.json
■■■ tests/test_basic.py
```

8. Key Concepts

- RAG: Retrieval-Augmented Generation — retrieves real docs before generating, reducing hallucination.
- Deflection Rate: % of queries resolved by bot without human escalation. Target: 70–85%.
- Confidence Score: Self-reported model certainty (0–1). Drives routing: answer, warn, or escalate.
- L0 vs L1: L0 = fully automated chatbot. L1 = human agent. This project bridges both.
- Citation Grounding: Every answer references KB article + section — builds trust, aids auditing.
- Single-File Prototype: Entire demo in index.html — maximises portability, zero setup on demo day.

9. Escalation Flow

Trigger: confidence < 0.55 → Ticket form auto-opens with issue summary, detected topic/category, partial AI answer, and timestamp pre-filled. User selects priority (Low / Medium / High / Critical) and submits.

10. Submission Checklist

Status	Item	Notes
■ Done	Working prototype	index.html — open immediately, no setup
■ Done	GitHub-ready structure	src/, data/, tests/ folders in place
■ Done	4 KB articles (16+ sec)	Markdown files in data/kb_articles/
■ Done	README + architecture	Diagram, quick-start, tech table
■ Done	Sample output JSON	outputs/sample_output.json
■ TODO	Demo video	2-min walkthrough — still pending
■ TODO	Team resumes	Add PDF resumes to repo root

Demo Tip: Show (1) high-confidence answer with citation → (2) low-confidence auto-escalation → (3) live dashboard deflection rate updating. Three moments cover all key features.